

Chapter Two

Basic elements of C++

What is C++?

- C++ is a **high-level, general-purpose programming language**.

It is an extension of the C language with added features like **object-oriented programming (OOP)**.

- Used for **system/software development, games, AI, IoT, and competitive programming**.

General Structure of a C++ Program

// 1. **Preprocessor Directives**

```
#include <iostream> //
```

2. **Namespace Declaration**

```
using namespace std;
```

// 3. **Global Declarations (optional)**

```
int x = 10;
```

// 4. **Function Declarations (Prototypes)**

```
void display(); /
```

// 5. **Main Function** — the starting point of every C++ program

```
int main() {
```

```
    // Local variable declarations
```

```
    int num = 5;
```

```
    // Statements (code instructions)
```

```
    cout << "Number: " << num << endl;
```

```
    // Function call
```

```
    display();
```

```
    return 0; // Program ends successfully
```

```
}
```

// 6. **User-defined Function Definitions**

```
void display()
```

```
{ cout << "Hello from function!" << endl; }
```

First program in C++

```
1. // the first C++ program
2. #include <iostream.h>
3. int main()
4. {
5.     cout << "Hello World!\n";
6.     return 0;
7. }
```

C++ program structure

- Every C++ program has a number of components
 - Comments
 - Compiler directive
 - *main* function
 - Braces
 - Statement

C++ program structure

- Comments
 - Are remarks that are ignored by the compiler .
 - Use comments to
 - Explain the purpose of a program
 - Keep notes regarding change to the source code
 - Store the names of programmers for future reference
 - Explain the parts of your program
 - comment:
 - Begin with a double slash and may appear anywhere in the program.

C++ program structure

- Compiler directives
 - Are instructions to the compiler rather than C++ instructions that instruct the C++ compiler to load a file from disk into the current program.
 - Begin with #
 - Eg. `#include <iostream.h>`
 - Tells preprocessor to include the input/output stream header file `<iostream.h>`
 - `#include <math.h>`: to manipulate numbers
 - `#include <string.h>`: to manipulate strings
 - `#define` : Defines a constant
- The **main** function
 - Is a function that runs first in C++ programs.
 - Program execution starts by executing whatever instructions found in the main function.
 - Is a function called by the operating system and returns a value to the operating system. The `int` keyword allows the main function to return an integer value. The return statement is what actually returns the value to the operating system.
 - Every C++ program must have only one main function .
 - The parentheses `{() }` following the main tell the compiler that main is a

C++ program structure

- Braces
 - Are used to mark the beginning and the end of a block of code.
 - Every opening brace must have a corresponding closing brace.
- Statements
 - Are instructions and commands that make the computer work.
 - Each statement must end in semicolon (;).
 - The semicolon tells the compiler that the statement is completed.

C++ program structure

- C++ is case sensitive because it interprets uppercase and lowercase letters differently
- For example
 - Cow and cow are two different combination of letters.
- White space
 - Blank lines, space characters and tabs
 - Used to make programs easier to read
 - Ignored by the compiler

Cont..

- Preprocessor directives
 - Processed by preprocessor before compiling
 - Begin with #
 - Example
 - `#include <iostream>` `// for cout, cin`
 - `#include <iomanip>` `// for setw, setprecision`
 - `#include <string>` `// for string class`
 - `#include <cmath>` `// for sqrt(), pow()`
 - `#include <fstream>`: to process files using `ifstream`, `ofstream`, `fstream`
 - `#include<ctime>`: for Time and date functions (`time()`, `clock()`).

Cont..

- Function `main`
 - A part of every C++ program
 - Exactly one function in a program must be `main`
 - what will be executed when you run your program
 - “main” is function where execution starts
 - Can “return” a value
 - Example
 - `int main()`
 - This `main` function returns an integer (whole number)
 - Body is delimited by braces (`{}`)
- Statements
 - Instruct the program to perform an action
 - All statements end with a semicolon (`;`)

Cont..

- Stream **insertion operator** (<<) : **to** sends) data into the output stream.
 - Value to right (right operand) inserted into left operand
 - Example
 - `cout << "Hello World!\n";`
 - Inserts the string " *Hello World!* " into the standard output
 - » Displays to the screen
- Stream **extraction operator** (>>) : **To** Extracts (reads) data from the input stream.
 - `Cin>> name,`
 - `Cin>>age`
- Escape characters: A character preceded by "\"
 - Example: "\"`\n`“ for new line, “`\t`” for tab, “`\r`” for Carriage return, `\b` for Backspace, etc.

Cont..

- **return** statement
 - One of several means to exit a function
 - When used at the end of **main**
 - The value 0 indicates the program terminated successfully
 - Example
 - `return 0;`

Basic Elements

- Five kind of tokens in C++
 - Comments
 - Keywords (reserved words)
 - Identifiers
 - Literals
 - Operators

Comments

- Remark about programs
- Explain programs to other programmers
 - Improve program readability
- Ignored by compiler
- Two types
 - Single-line comment
 - Begin with `//`
 - Example
 - `// This is a text-printing program.`
 - Multi-line comment
 - Start with `/*`
 - End with `*/`
- Typical uses
 - Identify program and who wrote it
 - Record when program was written
 - Add descriptions of modifications

Example

```
1  /* Fig. 2.1: fig02_01.cpp
2  Text-printing program. */
3  #include <iostream> // allows program to output data to the screen
4
5  // function main begins program execution
6  int main()
7  {
8      cout << "Welcome to C++!\n"; // display message
9
10     return 0; // indicate that program ended successfully
11
12 } // end function main
```

Welcome to C++!

Keywords (reserved words)

- Words with special meaning to the compiler
- Have a predefined meaning that can't be changed
- All reserved words are in lower-case letters
- Must not be used for any other purposes
- Example
 - *and, auto, bool, break, char, const, default, delete, do, double, else, for, friend, new, if, while, switch, case, class, private, public, void, int, string, return, float, double, sizeof, enum, inline, true, false, nullptr, struct union, try, catch, etc.*

Continued ...

asm	auto	bool	break	case	catch
const_cast	class	const	char	continue	default
dynamic_cast	do	double	delete	else	enum
explicit	extern	false	float	for	friend
goto	if	inline	int	long	mutable
namespace	new	operator	private	protected	public
reinterpret_cast	register	return	short	signed	sizeof
static_cast	static	struct	switch	template	this
throw	true	try	typedef	typeid	typename
union	unsigned	using	virtual	void	volatile
wchar_t					

Identifiers

- Programmer given names
- Identify classes, variables, functions, etc.
- Consist of letters, digits, and the underscore character (`_`)
- Must begin with a letter or underscore
- *Not be a reserved word*
- C++ is case sensitive
- Some predefined identifiers are `cout` and `cin`
- Unlike reserved words, predefined identifiers may be redefined, but it is not a good idea

Legal and Illegal Identifiers

- The following are legal identifiers in C++:
 - `first`
 - `conversion`
 - `payRate`

TABLE 2-1 Examples of Illegal Identifiers

Illegal Identifier	Description
<code>employee Salary</code>	There can be no space between <code>employee</code> and <code>Salary</code> .
<code>Hello!</code>	The exclamation mark cannot be used in an identifier.
<code>one + two</code>	The symbol <code>+</code> cannot be used in an identifier.
<code>2nd</code>	An identifier cannot begin with a digit.

Literals

- Explicit (constant) value that is used by a program
- Literals can be digits, letters or others that represent constant value to be stored in variables
 - Assigned to variables
 - Used in expressions
 - Passed to methods
- E.g.
 - `Pi = 3.14;` *// Assigned to variables*
 - `C= a * 60;` *// Used in expressions*
 - `char grade = 'A';`
 - `string name = "Ali";` *// "Ali" is a string literal*
 - `bool pass = true;`

Variables

- Location in memory where value can be stored
- All variables have two important attributes
 - A type - Once defined, the type of a C++ variable cannot be changed
 - A value - can be changed by assigning a new value
 - E.g. `int a = 5;`
 - Type integer
 - Value 5

Variable Declaration

- defining (creating) a variable
- Two parts
 - *Variable name* - unique name of the memory location
 - *Data type* (or *type*) - defines what kind of values the variable can hold
- Syntax
 - `<type> <Var-idf>;`
 - E.g.
 - `double varTime;`
 - `int myAge;`
- Can declare several variables of same type in one declaration
 - Comma-separated list
 - `int integer1, integer2, sum;`
- Variables must be declared before used

Rules for Naming Variables ^{Cont.}

- Must start with a letter or underscore (_)
- Can contain letters, numbers, and underscores
- Cannot start with a number
- Cannot use spaces or reserved keywords
- Variable names
 - Valid identifier
 - Series of characters (letters, digits, underscores)
 - Cannot begin with digit
 - Case sensitive
- Assignment operator =
 - Assigns value on left to variable on right
 - Binary operator (two operands)
 - Example:
 - `sum = variable1 + variable2;`
 - Add the values of `variable1` and `variable2`
 - Store result in `sum`

```

1 // Fig. 2.5: fig02_05.cpp
2 // Addition program that displays the sum of two numbers.
3 #include <iostream.h> // allows program to perform input and output
4
5 // function main begins program execution
6 int main()
7 {
8     // variable declarations
9     int number1; // first integer to add
10    int number2; // second integer to add
11    int sum; // sum of number1 and number2
12
13    cout << "Enter first integer: "; // prompt user for data
14    std::cin >> number1; // read first integer from user into number1
15
16    cout << "Enter second integer: "; // prompt user for data
17    cin >> number2; // read second integer from user into number2
18
19    sum = number1 + number2; // add the numbers; store result in sum
20
21    cout << "Sum is " << sum << std::endl; // display sum; end line
22
23    return 0; // indicate that program ended successfully
24
25 } // end function main

```

```

Enter first integer: 45
Enter second integer: 72
Sum is 117

```

Variables and Memory Concept

- Variable names
 - Correspond to actual locations in computer's memory
 - Every variable has name, type, size and value
 - When new value placed into variable, overwrites old value
 - Writing to memory is destructive
 - Reading variables from memory nondestructive
 - age, price, and name are example of valid variables
 - Example
 - `int number1= 45;`
 - `int number2= 72;`
 - `int sum = 0;`
 - `sum = number1 + number2;`
 - » Value of `sum` is overwritten
 - » Values of `number1` and `number2` remain intact

Memory locations after calculating and storing the sum of number1 and number2.

number1	45
number2	72
sum	117

- Types of variable/scope of variable:
 - **Global variable**: accessible through out the program.
 - **Local** : accessable only inside a function or a block.
 - `#include<iostream.h>`
 - `Int x,y;    //x and y are global variable`
 - `Int main()`
 - `{    int sum=0;    // sum is local variable`
 - `sum=x+y;`
 - `}`

Data Types

- When you define a variable in C++, you must tell the compiler what kind of variable it is
 - Tell the data type
 - **how much memory** a variable needs to store a value.
- Data Type: set of values together with a set of operations
- C++ data can be classified into three categories:
 - Simple/Primitive/basic data type: built-in types.
 - E.g. int, float, double, char, and bool
 - Structured data type: derived +user defined types
 - array, reference (&), structure, class, union, and enum.
 - Pointers: stores the **memory address** of another variable

Simple Data Types

- Three categories of simple data
 - Integral: integers (numbers without a decimal)
 - Floating-point: decimal numbers
 - Enumeration type: user-defined data type

Primitive data types

Type	Size	Values
<code>unsigned short int</code>	2 bytes	0 to 65,535
<code>short int(signed short int)</code>	2 bytes	-32,768 to 32,767
<code>unsigned long int</code>	4 bytes	0 to 4,294,967,295
<code>long int(signed long int)</code>	4 bytes	-2,147,483,648 to 2,147,483,647
<code>int</code>	2 bytes	-32,768 to 32,767
<code>unsigned int</code>	2 bytes	0 to 65,535
<code>signed int</code>	2 bytes	-32,768 to 32,767
<code>char</code>	1 byte	256 character values
<code>float</code>	4 bytes	3.4e-38 to 3.4e38
<code>double</code>	8 bytes	1.7e-308 to 1.7e308
<code>long double</code>	10 bytes	1.2e-4932 to 1.2e4932

Simple Data Types

- Type int
 - represent integers or whole numbers
 - Some rules to follow:
 - Plus signs do not need to be written before the number
 - Minus signs must be written when using negative #'s
 - Decimal points cannot be used
 - Commas cannot be used
 - Leading zeros should be avoided (octal or base 8 #'s)
 - Signed - negative or positive
 - Unsigned - positive
 - Short
 - Long

Simple Data Types

- Type double, float
 - used to represent real numbers
 - many programmers use type float
 - avoid leading zeros, trailing zeros are ignored
 - long double

Simple Data Types

- Type char
 - used to represent character data
 - a single character which includes a space
 - 1 byte, enough to hold 256 values
 - must be enclosed in single quotes eg. 'd'
 - char grade = 'A';
 - Escape sequences treated as single char
 - '\n' newline
 - '\'' apostrophe
 - '\"' double quote
 - '\t' tab
 - (0-255) or as a member of the ASCII set
 - E.g. the lowercase letter "a" is assigned the value 97

Simple Data Types

- Strings
 - used to represent textual information
 - Are sequence of characters under “ “.
 - string constants must be enclosed in double quotation marks eg. “Hello world!”
 - empty string “”
 - new line char or string “\n”
 - “the word \”hello\”” (puts quotes around “hello”)
 - String variables use:
#include “apstring.cpp”
 - use quotes for user supplied libraries

Area.cpp

```
//C++ program to compute area of a rectangle
#include <iostream.h>

int main() {
    float Length;
    float Width;
    // Extract length and width
    cout << "enter width and length: ";
    cin >> Length >> Width;
    // Compute and insert the area
    float Area = Length * Width;
    cout << "Area = " << Area << " = Length "
        << Length << " * Width " << Width <<
    endl;
    return 0;}
```

Operator

- symbols that take one or more arguments (operands) and operates on them to produce a result
- A *unary operator* requires *one* operand.
 - E.g. Increment (++) and decrement (--), !(negation)
- A *binary operator* requires *two* operands.
 - E.g. +, -, *, /, %, etc.
- A ternary operator requires three operands.
- 5 Operators
 - Arithmetic operators
 - Assignment operator
 - Increment/Decrement operators
 - Relational operators
 - Logical operators

Arithmetic Operators

- Common
 - Addition +
 - Subtraction -
 - Multiplication *
 - Division /
 - Mod %
- Note
 - No exponentiation operator
 - Single division operator
 - Operators are overloaded to work with more than one type of object

Integer Division

- Integer division produces an integer result
 - Truncates the result
- Examples
 - $3 / 2$ evaluates to 1
 - $4 / 6$ evaluates to 0
 - $10 / 3$ evaluates to 3

Mod

- Produces the remainder of the division
- Examples
 - $5 \% 2$ evaluates to 1
 - $12 \% 4$ evaluates to 0
 - $4 \% 5$ evaluates to 4

Arithmetic Operators and Precedence

- Consider $m * x + b$ which of the following is it equivalent to
 - $(m * x) + b$
 - $m * (x + b)$
- Operator precedence tells how to evaluate expressions
- Standard precedence order
 - $()$ Evaluate first, if nested innermost done first
 - $* / \%$ Evaluate second. If there are several, then evaluate from left-to-right
 - $+ -$ Evaluate third. If there are several, then evaluate from left-to-right

Operator precedence in C++

Precedence	Operators	Associativity	Example
1 (Highest)	:: (scope resolution)	Left-to-right	std::cout
2	(), [], ->, . (member access), function call	Left-to-right	arr[0], obj.method()
3	++ -- (postfix)	Left-to-right	x++, y--
4	++ -- (prefix), + - (unary), !, ~, * (dereference), & (address)	Right-to-left	++x, -a, !flag
5	* / %	Left-to-right	a*b, c/d
6	+ -	Left-to-right	a+b-c
7	<< >>	Left-to-right	a << 2, b >> 1
8	< <= > >=	Left-to-right	x < y
9	== !=	Left-to-right	x == y
10	&	Left-to-right	x & y
11	^	Left-to-right	x ^ y
12	`	`	Left-to-right
13	&&	Left-to-right	a && b
14	,		,
15	?: (ternary conditional)	Right-to-left	(x>0)?1:0
16	= += -= *= /= %= &= `	= ^= <<= >>= `	Right-to-left

Arithmetic Operator Precedence

- Examples

20 - 4 / 5 * 2 + 3 * 5 % 4

(4 / 5)

((4 / 5) * 2)

((4 / 5) * 2) (3 * 5)

((4 / 5) * 2) ((3 * 5) % 4)

(20 - ((4 / 5) * 2)) ((3 * 5) % 4)

(20 - ((4 / 5) * 2)) + ((3 * 5) % 4)

Increment & Decrement Operators

- Increment operator: increment variable by 1
- Decrement operator: decrement variable by 1
 - Pre-increment: `++variable`
 - Post-increment: `variable++`
 - Pre-decrement: `--variable`
 - Post-decrement: `variable--`

Increment & Decrement Operators (continued)

- `++count;` or `count++;` increments the value of `count` by 1
- `--count;` or `count--;` decrements the value of `count` by
- If `x = 5;` and `y = ++x;`
 - After the second statement both `x` and `y` are 6
- If `x = 5;` and `y = x++;`
 - After the second statement `y` is 5 and `x` is 6

Assignment Operator

- Assignment operator =
 - Assign the **value on the right-hand side** to the **variable on the left-hand side**.
 - Binary operator (two operands)
 - Example:
 - `int a= 5;`
 - `float b= 9.66;`
 - `char ch='d';`
 - `int m, n, p;`
 - `m = n = p = 100;`
 - Compound Assignment Operators
 - E.g `+=`, `-=`, `*=`, `/=`, `%=`, etc.
 - `sum+=x`, `a += 3`, `b=-2`;
 - `Sum+=x` means `sum=sum+x`
 - `A/=5` means `A=A/5`

Relational Operators

Operator	Name	Example
==	Equality	5 == 5 // gives 1
!=	Inequality	5 != 5 // gives 0
<	Less Than	5 < 5.5 // gives 1
<=	Less Than or Equal	5 <= 5 // gives 1
>	Greater Than	5 > 5.5 // gives 0
>=	Greater Than or Equal	6.3 >= 5 // gives 1

Relational operators

Logical Operators

- Like the relational operators, logical operators evaluate to 1 or 0.

Operator	Name	Example
!	Logical Negation	!(5 == 5) // gives 0
&&	Logical And	5 < 6 && 6 < 6 // gives 1
	Logical Or	5 < 6 6 < 5 // gives 1
Logical operators		

Cont..

- *Example*

- *!20* *// gives 0*

- *10 && 5* *// gives 1*

- *10 || 5.5* *// gives 1*

- *10 && 0* *// gives 0*

-

Type conversion/ casting

- A value in any of the built-in types can be converted
 - Called type cast
- Syntax
 - (`<data – type>`)value; or `<data – type>` (value);
- E.g.
 - (int) 3.14 // converts 3.14 to an int to give 3
 - (long) 3.14 // converts 3.14 to a long to give 3L
 - (double) 2 // converts 2 to a double to give 2.0
 - (char) 122 // converts 122 to a char whose code is 122
 - (unsigned short) 3.14 // gives 3 as an unsigned short

- Some times the compiler does the type casting – implicit type cast
- E.g
 - `double d = 1; // d receives 1.0`
 - `int i = 10.5; // i receives 10`
 - `i = i + d;`
- Converting using `static_cast()`
 - syntax: `static_cast<Type>(value)`
 - `int a = 7;`
 - `double b = static_cast<double>(a);`
 - `cout << b; // 7.0`

Expression

- an expression is any combination of variables, constants, operators, and function calls that produces a value.
- Expression is segments of code that perform computations and return values .
- Expressions can contain: Example:
 - a number literal, 3.14
 - a variable, count
 - a function call, sum(x, y)
 - an operator between two expressions (binary operator),
a + b, x > y, a && b,
 - an operator applied to one expression (unary operator),
-discount, b = 10, x--
 - expressions in parentheses.

(3 14-amplitude)

Statements

- Roughly equivalent to sentences in natural languages
- Forms a complete unit of execution.
- terminating the expression with a semicolon (;)
- Three kinds of statements in java
 - expression statements
 - declaration statements
 - control flow statements
- expression statements
 - Null statements
 - Assignment expressions
 - Any use of ++ or --
 - Function calls
 - Object creation expressions